

Roadmap del Proyecto

TFG DAM — Stockly

Adrián Bravo Santos · Miguel Ángel Florido
Generado el 2026-05-16

Roadmap / Itinerario de tareas

Lista priorizada de mejoras de **aspecto** y **funcionalidad** de la aplicación, agrupadas por fases y con responsable orientativo (A = Adrián Bravo Santos, M = Miguel Ángel Florido, AM = ambos). Las marcas reflejan el estado real al final del Sprint 4.

Leyenda:  hecho ·  en curso ·  pendiente

Fase 1 — Núcleo funcional (Sprints 0-2)

#	Tarea	Responsable	Estado
1.1	Modelado E/R + esquema SQL + 500 productos semilla	M	
1.2	API REST: productos, categorías, reservas	A	
1.3	Frontend SPA básico (catálogo + reservar)	A	
1.4	Reservas concurrentes con FOR UPDATE	AM	
1.5	Login + registro + JWT + bcrypt	A	
1.6	Roles cliente / operario / admin en API	A	

Fase 2 — Mejora de UX y panel administrativo (Sprint 3)

#	Tarea	Responsable	Estado
2.1	Sistema de diseño (tokens CSS, espaciados, tipografía)	A	
2.2	Cambio de estado de reservas (confirmar / entregar / cancelar)	A	
2.3	Tabla movimientos y auditoría automática	M	
2.4	Endpoint /api/admin/stats con KPIs	M	
2.5	Vista Dashboard con gráficos en CSS	M	
2.6	Vista Inventario con CRUD + alerta stock bajo	A	
2.7	Vista Usuarios (alta/edición admin)	M	
2.8	Export CSV de reservas	M	

Fase 3 — PWA, móvil y modo oscuro (Sprint 4)

#	Tarea	Responsable	Estado
3.1	Manifest + iconos	A	
3.2	Service Worker (cache shell, offline-first del esqueleto)	A	
3.3	Bottom navigation en móvil	A	
3.4	Modo oscuro con preferencia del sistema + toggle	A	
3.5	Diseño responsive: grid 2 columnas en móvil	A	
3.6	Toasts y modales accesibles	A	

Fase 4 — Despliegue y documentación (Sprint 5, en curso)

#	Tarea	Responsable	Estado
4.1	Despliegue del backend en VPS/Render (ver hosting.md)	A	
4.2	Compra y configuración de dominio	A	
4.3	HTTPS con Let's Encrypt	A	
4.4	Backup automatizado de la BD	M	
4.5	Workflow GitHub Actions: build + deploy automático	A	
4.6	Memoria PDF	AM	
4.7	Vídeo demo 3 min	AM	
4.8	Diagramas: E/R, casos de uso, despliegue	M	

Fase 5 — Mejoras de aspecto (UI/UX) ⌚

#	Tarea	Prioridad
5.1	Skeleton loaders mientras se cargan listados	media
5.2	Animaciones sutiles entre vistas (View Transitions API)	baja
5.3	Imágenes reales de producto + lazy loading	media
5.4	Modo "alta densidad" para inventario en pantallas grandes	baja
5.5	Toast con stack (varios mensajes apilables)	baja
5.6	Gráficos avanzados con Chart.js (donut categorías, line)	media
5.7	Filtros guardados en URL (compatibles)	media
5.8	Internacionalización es / en	baja
5.9	Atajos de teclado en escritorio (/ para buscar, n nuevo)	baja
5.10	Accesibilidad WCAG AA: foco visible, ARIA, contraste	alta

Fase 6 — Mejoras de funcionalidad ⌚

#	Tarea	Prioridad
6.1	Carrito: reservar varios productos a la vez	alta
6.2	Notificaciones (Web Push) cuando una reserva se confirma	media
6.3	Lector de códigos QR/barras (cámara) para encontrar producto	alta
6.4	Historial de cambios de stock por producto (drilldown)	media
6.5	Gestión de proveedores y entradas de mercancía	media
6.6	Caducidades y lotes	baja
6.7	Subida de imágenes de producto (multer + carpeta uploads/)	alta
6.8	Exportación PDF de albarán al entregar una reserva	media
6.9	Multi-almacén (almacenes con sus propias ubicaciones)	baja
6.10	Recuperación de contraseña por email	media

Fase 8 — App móvil para empleado (companion Android) ⌚

App nativa-ligera basada en **Capacitor** que envuelve la web existente y la presenta como aplicación instalable, con foco en el flujo del operario de almacén: ver reservas pendientes, confirmar entregas y reportar incidencias desde el propio puesto, sin abrir el navegador.

#	Tarea	Prioridad
8.1	Bootstrap del proyecto Capacitor (<code>mobile/</code>) apuntando al dominio HTTPS	alta
8.2	Vista "Mis reservas asignadas" (filtrada por operario logueado, agrupada por ubicación)	alta
8.3	Pantalla detalle de reserva con ubicación en almacén (pasillo / estantería)	alta
8.4	Acción " Confirmar entrega " con foto opcional + firma + cambio de estado	alta
8.5	Acción " Dar incidencia " (rotura, faltante, mal estado) con descripción y fotos	alta
8.6	Endpoints backend nuevos: <code>PATCH /reservas/:id/entregar</code> , <code>POST /reservas/:id/incidencias</code>	alta
8.7	Tabla <code>incidencias</code> (id, reserva_id, tipo, descripcion, fotos[], operario, fecha)	alta
8.8	Login con biometría / sesión persistente (<code>@capacitor/biometric-auth</code>)	media
8.9	Push notifications al asignar reserva (OneSignal o FCM)	media
8.10	Lector de QR/barras integrado para localizar producto al confirmar	media
8.11	Generación de APK firmada + canal de distribución interna (TestFlight/PlayConsole)	baja
8.12	Modo offline: cola local de confirmaciones cuando no hay red, sync al recuperar	baja

Justificación: el operario de almacén pasa el turno con el móvil en la mano. Una app específica (en lugar de la web genérica) reduce la fricción a "abrir → ver lista → tocar entregar". La parte de incidencias añade trazabilidad real: hoy las roturas se anotan en papel y se pierden.

Fase 7 — Calidad de código y operaciones

#	Tarea	Prioridad
7.1	Tests unitarios backend (Jest)	alta
7.2	Tests E2E (Playwright) sobre login y reserva	alta
7.3	ESLint + Prettier	media
7.4	Validación con Zod / Joi en API	media
7.5	Migraciones versionadas (Knex / Prisma migrate)	media
7.6	Logging estructurado (pino) y rotación	baja
7.7	Monitorización (Uptime Kuma / Better Stack)	baja

Itinerario sugerido para terminar el TFG (8 semanas)

Sem 1 ▶ 4.1 + 4.2 + 4.3 (despliegue + dominio + HTTPS)
Sem 2 ▶ 5.10 (accesibilidad) + 6.1 (carrito)
Sem 3 ▶ 6.3 (escáner QR) + 5.1 (skeletons)
Sem 4 ▶ 6.7 (subida imágenes) + 5.3 (lazy load)
Sem 5 ▶ 7.1 + 7.2 (tests)
Sem 6 ▶ 4.8 (diagramas) + 4.5 (CI/CD)
Sem 7 ▶ 4.6 (memoria) + 4.7 (vídeo)
Sem 8 ▶ Defensa: ensayo + slides

Post-defensa / continuación: Fase 8 (companion móvil del operario) como evolución natural — añade valor de negocio real y demuestra extensión de la arquitectura sin reescritura.